Answer to Question No. 01

AI Engineering Overview

Core Difference Between Agentic AI and Generative AI

Generative AI mainly focuses on generating content from prompts given by users. It can create text, images, code, audio, or videos, but normally it only responds to instructions and does not independently perform tasks.

Agentic AI is more advanced because it can take actions by itself. It can plan tasks step by step, remember previous context, use tools or APIs, and complete a goal with less human guidance.

Real-World Business Use Case

For example, an online shopping company may use Generative AI to write product descriptions or automatically reply to customer messages.

On the other hand, Agentic AI can work like a smart assistant. If a customer complains about a delayed order, the AI can check the order status, create a support ticket, send an apology email, and update the customer automatically without needing human support every time.

So, the main difference is:

Generative AI creates content.

Agentic AI can make decisions and perform actions.

Answer to Question No. 02

In Local AI Development, both CPU and GPU can run AI models, but their performance is very different. A CPU is good for normal computing tasks and small AI projects, but when running large AI models or LLMs, it becomes slower because it processes tasks step by step.

A GPU is much faster for AI work because it can handle many calculations at the same time. This is important for model training, inference, image processing, and running local LLMs smoothly. For example, when running a local chatbot model, CPU may generate responses slowly, while GPU gives faster output and better performance.

Docker is also important in AI development because it helps create a separate and stable environment for projects. Sometimes different AI tools need different library versions, and without Docker, there can be dependency problems. Docker solves this by packaging everything together, so the project works the same on every system.

It also makes deployment easier and helps developers manage AI applications more efficiently.

Answer to Question No.03

Quantization methods are techniques used to reduce the size and hardware requirements of Large Language Models (LLMs). Normally, LLMs use high-precision data formats like FP32 or FP16, which require a lot of RAM and GPU memory. Quantization converts these model weights into lower precision formats such as INT8 or 4-bit.

This is very useful when running local LLMs on personal computers because many users do not have high-end GPUs. After quantization, the model becomes smaller and can run on lower hardware with acceptable performance.

Quantization also helps in latency optimization because smaller models process data faster and generate responses more quickly. In addition, it improves memory management by reducing RAM and VRAM usage. For example, a model that normally needs 16GB VRAM may run on 8GB after quantization.
Popular quantization formats used in local AI are GGUF, GPTQ, and AWQ. These methods are commonly used with tools like Ollama and LM Studio for running local AI models efficiently.

Answer to Question No. 04

In production-level AI applications, System Prompt Design is very important because it controls how the AI should behave, respond, and follow instructions. A well-designed system prompt helps the AI stay focused on its task, maintain a proper tone, and avoid giving unrelated answers.
For example, in a customer support chatbot, the system prompt can instruct the AI to answer politely, avoid sensitive topics, and only provide information related to company services. Without a proper system prompt, the AI may generate confusing or incorrect responses.

Guardrails and Safety are also critical because AI systems can sometimes produce harmful, biased, false, or unsafe outputs. In real business environments, this can create security risks or damage company reputation.

Guardrails help limit unsafe behavior by filtering harmful content, protecting private data, and preventing misuse of the system. They also reduce hallucinations and improve reliability.

So, both system prompts and safety guardrails are necessary to make AI applications more secure, stable, and trustworthy for real users.

Answer to Question No. 05

RAG (Retrieval-Augmented Generation) is an AI architecture that improves the performance of Large Language Models by combining information retrieval with text generation. Instead of relying only on the model's internal knowledge, RAG first retrieves relevant information from external documents and then uses that information to generate accurate answers.

In this system, when a user asks a question, the query is first converted into an embedding. Then the system searches a vector database to find the most relevant document sections. After retrieving this information, it is sent to the language model, which generates the final response based on both the question and the retrieved context.

Document Processing plays an important role in preparing raw data for the system. It includes cleaning text, removing unnecessary data, extracting useful content, and converting documents like PDF or Word files into machine-readable format.

Chunking Strategy is also very important because large documents are divided into smaller pieces called chunks. This helps the system retrieve only the most relevant part of the document instead of processing the entire file. Good chunking improves accuracy, reduces token usage, and increases response speed.

So, RAG helps AI systems give more accurate, updated, and reliable answers by combining document retrieval with generation.